INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DO SUDESTE DE MINAS GERAIS - *CAMPUS* MURIAÉ
GESTÃO DA TECNOLOGIA DA INFORMAÇÃO

Alexandre do Amaral Quintão
Sabrina Caetano Fontes

AURIA

sistema de gestão de manutenção predial e ativos

API REST em ASP.NET Core integrada ao aplicativo Flutter

Muriaé - MG
2026

SUMÁRIO

1 INTRODUÇÃO 1
1.1 Público-Alvo 1
2 DESCRIÇÃO GERAL DO PRODUTO 2
2.1 Descrição do Cliente 2
2.2 Descrição Geral do Produto 3
***2.2.1 Situação Atual da Empresa* 4**
***2.2.2 Escopo do Produto - Módulos e Funcionalidades* 5**
***2.2.3 Arquitetura da Solução* 6**
2.3 Premissas 7
3 REQUISITOS 9
3.1 Requisitos Funcionais 9
3.2 Requisitos Não Funcionais 11
3.3 Regras de Negócio 13
4 PLANO DE LIBERAÇÕES 14

1 INTRODUÇÃO

Este documento de requisitos tem como objetivo definir e descrever as funcionalidades e características do sistema desenvolvido, denominado Auria, um sistema de gestão de manutenção predial e ativos. O Auria é a evolução do sistema Aura: a versão anterior funcionava apenas no aplicativo Flutter, com os dados guardados no próprio aparelho, e a nova versão adota uma arquitetura cliente-servidor, na qual o aplicativo consome a **AuriaAPI**, uma API REST desenvolvida em ASP.NET Core.

A AuriaAPI concentra as regras de negócio, o acesso ao banco de dados MySQL, a autenticação por JWT e o cache, enquanto o aplicativo Flutter fica responsável pela interface com o usuário. Este documento servirá como guia para os desenvolvedores, ajudando-os a compreender os requisitos essenciais, a arquitetura adotada e as expectativas do cliente.

1.1 Público-Alvo

O público-alvo deste documento de especificação inclui:

1. **Equipe de Desenvolvimento:** Desenvolvedores (Flutter e Backend .NET), designers de UI/UX e testadores responsáveis pela implementação física e validação do sistema.
2. **Gestores de Projeto:** Gerentes de projeto, engenheiros de software e líderes técnicos que supervisionam o cronograma e garantem o cumprimento dos requisitos.
3. **Clientes e Stakeholders:** Diretores de operações, gestores de *facilities* e patrocinadores do projeto que precisam validar se o escopo atende às necessidades estratégicas da empresa.

2 DESCRIÇÃO GERAL DO PRODUTO

2.1 Descrição do Cliente

Durante o processo de levantamento de requisitos, foi identificado que o cliente realizava o controle de manutenção de ativos e infraestrutura de forma manual, utilizando papel ou planilhas no Excel. Essa abordagem se mostrou limitada frente às necessidades crescentes de agilidade, organização, controle de custos e extensão da vida útil dos equipamentos. Por esse motivo, o cliente manifestou interesse na adoção de um sistema automatizado de gestão de manutenção predial e ativos.

A primeira versão do sistema (Aura) atendeu ao cadastro e ao acompanhamento das manutenções, mas guardava os dados apenas no aparelho de cada usuário. Com isso, as informações não eram compartilhadas entre o solicitante, o técnico e o gestor, e cada aparelho tinha uma cópia diferente dos dados. O cliente passou então a demandar uma plataforma centralizada, em que todos os perfis trabalhem sobre a mesma base de dados, com acesso seguro e controle de permissões.

A principal demanda continua sendo uma plataforma digital integrada, que permita o cadastro ágil de ativos físicos e o mapeamento dos ambientes (salas, blocos e andares), com registro do estado de conservação, da data de aquisição, do prazo de garantia e da vida útil estimada. Qualquer usuário autorizado deve conseguir abrir um chamado de forma simples, informando o equipamento, o tipo de manutenção (corretiva, preventiva ou preditiva) e a prioridade, e acompanhar o andamento até a conclusão.

Do ponto de vista administrativo, o cliente solicitou o controle do estoque de peças integrado às ordens de serviço, alertas de reposição, o registro de tempo e custos de cada atendimento e um painel de indicadores com tempo médio de atendimento, custos e ativos com mais falhas. Também foi pedido que o sistema seja acessível a colaboradores com deficiência.

Aqui estão as principais funcionalidades desejadas para o sistema de gestão de manutenção predial e ativos, com base nos requisitos do usuário:

- **Cadastro Centralizado de Ativos:** Permitir o registro completo de ativos físicos e ambientes, contendo dados de patrimônio, garantia, localização e vida útil estimada, em uma base única compartilhada por todos os usuários.
- **Gestão Descentralizada de Chamados:** Possibilitar a abertura rápida de chamados por usuários internos, com a descrição do problema, a prioridade e o tipo de manutenção.
- **Aprovação e Designação de Técnicos:** Permitir que o gestor aprove o chamado e designe o técnico responsável, gerando a ordem de serviço.
- **Notificações de Status:** Garantir o envio automático de notificações aos envolvidos sobre o andamento e a conclusão dos chamados.

- **Planejamento Preventivo:** Apresentar o cronograma das manutenções preventivas e alertar sobre ativos que precisam de atenção.
- **Rastreabilidade de Ordens de Serviço:** Registrar o técnico, o tempo gasto, as peças utilizadas, o parecer técnico e os custos de cada ordem de serviço.
- **Controle de Estoque Integrado:** Controlar o inventário de peças e debitar automaticamente o que for consumido nas ordens de serviço.
- **Alertas de Nível Mínimo:** Emitir avisos automáticos quando o estoque atingir a quantidade mínima de segurança.
- **Indicadores de Desempenho (KPIs):** Gerar um painel com taxa de conclusão, tempo médio de atendimento, custos e ativos com mais falhas, com exportação em PDF.
- **Segurança e Confidencialidade:** Proteger os dados com autenticação, senhas criptografadas e níveis de permissão por perfil.
- **Acessibilidade:** Permitir o uso do sistema por pessoas com deficiência visual, motora ou cognitiva.

2.2 Descrição Geral do Produto

O Sistema de Gestão de Manutenção Predial e Ativos, denominado **Auria**, é uma plataforma de software composta por duas partes integradas:

- **AuriaAPI:** API REST desenvolvida em .NET 9 / ASP.NET Core, organizada em Clean Architecture. É responsável pelas regras de negócio, pela persistência no MySQL, pela autenticação JWT, pelo cache e pelas notificações.
- **Aplicativo Auria:** aplicativo em Flutter/Dart que roda no Windows, na Web e no Android. Consome a API e apresenta uma interface própria para cada perfil de usuário.

O sistema tem como objetivo proporcionar um fluxo eficiente desde a abertura de chamados cotidianos até o planejamento preventivo da infraestrutura, garantindo alta disponibilidade dos ativos e controle rigoroso de custos. O fluxo principal de atendimento é:

1. O solicitante abre um chamado pelo aplicativo, e os administradores são notificados.
2. O administrador aprova o chamado e designa um técnico: a ordem de serviço é criada e o chamado passa ao status “Em Execução”.
3. O técnico registra as peças utilizadas, que são debitadas do estoque, e conclui a ordem com o tempo gasto e o parecer técnico.
4. O chamado passa a “Concluído” e o solicitante é notificado.

2.2.1 Situação Atual da Empresa

A empresa enfrenta desafios operacionais no gerenciamento de suas instalações e equipamentos. Além dos problemas originais do controle manual, a primeira versão do sistema revelou limitações técnicas que motivaram a criação da API. Abaixo estão os principais pontos que caracterizam a situação atual:

Processo Manual de Chamados

- Abertura de solicitações de reparo via e-mail, telefone ou formulários físicos, resultando em falhas de comunicação e esquecimentos.
- Falta de padronização na descrição dos problemas e na definição da prioridade das manutenções.
- Dificuldade de acompanhamento do status por parte do usuário solicitante.

Manutenções Majoritariamente Corretivas

- Ausência de um cronograma de revisões, gerando alta taxa de manutenções corretivas (reparos após a quebra).
- Paradas inesperadas de equipamentos críticos, prejudicando a produtividade geral da empresa.

Descontrole no Inventário Patrimonial e no Estoque

- Informações sobre localização, garantia e estado de conservação espalhadas em planilhas desatualizadas.
- Controle manual das peças de reposição, sem integração com os reparos realizados, gerando falta de peças essenciais ou excesso de itens sem giro.

Limitações da Primeira Versão (Aura)

- Dados guardados apenas no aparelho de cada usuário, sem compartilhamento entre solicitantes, técnicos e gestores.
- Regras de negócio, como a baixa de estoque e as notificações, executadas no próprio aplicativo, o que permitia inconsistências entre aparelhos.
- Ausência de autenticação centralizada e de controle de acesso por perfil no servidor.
- Ausência de indicadores consolidados, já que cada aparelho conhecia apenas uma parte dos dados.

2.2.2 Escopo do Produto - Módulos e Funcionalidades

Portal de Chamados e Execução de Ordens de Serviço (Aplicativo)

Este módulo é direcionado aos colaboradores da empresa (perfil Solicitante), para a abertura de solicitações, e aos técnicos de manutenção (perfil Técnico), que executam as ordens de serviço e registram o consumo de peças.

Funcionalidades	**Descrição**
Abertura de chamados em 3 etapas	O solicitante escolhe o equipamento, descreve o problema e a localização e define o tipo de manutenção e a prioridade.
Meus chamados com linha do tempo	O solicitante acompanha cada chamado (Aguardando aprovação, Em execução, Concluído ou Cancelado) e pode cancelar enquanto ele está aberto.
Meu dia do técnico	Painel com as ordens de serviço designadas ao técnico, ordenadas por prioridade.
Registro de peças utilizadas	O técnico informa as peças e quantidades usadas, e a API debita o estoque automaticamente.
Conclusão da ordem de serviço	O técnico registra o tempo gasto, o custo de mão de obra e o parecer técnico, e o chamado é concluído.
Notificações	Central de avisos com contador de não lidas no cabeçalho e no menu.

Tabela 1 – Funcionalidades do Portal de Chamados e Execução de Ordem de Serviço.

Painel Administrativo de Gestão Estratégica

O módulo gerencial é dedicado ao perfil Administrador, fornecendo controle sobre os cadastros, a aprovação de chamados, o almoxarifado de peças e os indicadores de desempenho.

Funcionalidades	**Descrição**
Painel geral	Visão do dia com pendências, chamados por status e atalhos para as ações mais comuns.
Quadro de chamados (kanban)	Chamados organizados por status; a aprovação e a designação do técnico são feitas em um painel lateral.
Cadastro de ambientes e ativos	Gerenciamento dos locais (bloco, andar e observações) e dos equipamentos, com patrimônio, estado de conservação, aquisição, garantia e vida útil.
Gestão de estoque	Controle de peças com quantidade, estoque mínimo e custo unitário, com destaque dos itens abaixo do mínimo.

Funcionalidades	Descrição
Preventivas	Cronograma diário das ordens preventivas e alertas de ativos em estado ruim, com mais de 2 anos de uso ou com garantia vencida.
Gestão de usuários	Cadastro de técnicos, solicitantes e administradores, com definição de perfil.
Indicadores (BI)	Taxa de conclusão, tempo médio de atendimento e de execução, custos de mão de obra e de peças e ativos com mais falhas, com exportação de relatório em PDF.

Tabela 2 – Funcionalidades do Painel Administrativo de Gestão.

Serviços da Plataforma (AuriaAPI)

Funcionalidades transversais executadas pela API, que dão suporte a todos os módulos.

Funcionalidades	Descrição
Autenticação e autorização	Login com e-mail e senha, geração de token JWT com validade de 8 horas e restrição de acesso por perfil.
Padronização das respostas	Todas as respostas, de sucesso ou de erro, seguem o formato ResultWrapper (`success`, `message`, `data`, `errors`).
Cache de consultas	Lista de ambientes (5 minutos) e indicadores (2 minutos) em cache na memória, com invalidação quando os dados mudam.
Notificações automáticas	Avisos gerados pela API para novo chamado, ordem designada, chamado em execução, cancelado ou concluído e estoque baixo.
Documentação interativa	Swagger em `/swagger` e roteiro de testes no arquivo `AuriaAPI.http`.
Administrador inicial e dados de demonstração	Criação automática do primeiro administrador a partir da configuração e, em ambiente de desenvolvimento, de dados de exemplo para testes.

Tabela 3 – Serviços oferecidos pela AuriaAPI.

2.2.3 Arquitetura da Solução

A AuriaAPI segue a Clean Architecture, dividida em quatro camadas. A regra de dependência é **Api** $\rightarrow$ **Application** $\rightarrow$ **Domain** $\leftarrow$ **Infra**: o núcleo (Domain) não conhece banco de dados, HTTP nem JWT, e a ligação entre interfaces e implementações acontece apenas no `Program.cs`, por injeção de dependência.

Camada	Responsabilidade
Domain	Entidades (Usuário, Ambiente, Ativo, Chamado, Ordem de Serviço, Item de Estoque, Notificação), regras próprias de cada entidade, constantes e interfaces dos repositórios.
Application	Casos de uso (services), DTOs com validação, mapeamentos, ResultWrapper e interfaces de serviços técnicos (cache, token, hash de senha).
Infrastructure	Entity Framework Core com MySQL, repositórios e Unit of Work, cache em memória, geração de JWT, BCrypt e criação do administrador inicial.
API (Presentation)	Controllers REST, middleware de tratamento de erros e configuração de JWT, Swagger, CORS e validação.

Tabela 4 – Camadas da AuriaAPI.

Componente	Tecnologias
API	.NET 9, ASP.NET Core Web API (C#), Entity Framework Core 9 com Pomelo (MySQL), JWT Bearer, BCrypt.Net-Next, IMemoryCache, Swagger/Swashbuckle e Data Annotations.
Banco de dados	MySQL 8, criado pela migration `InitialCreate`, com as tabelas usuário, ambiente, ativo, chamado, ordem de serviço, item de estoque, consumo de itens e notificação.
Aplicativo	Flutter/Dart (Windows, Web e Android), com `http`, `shared_preferences`, `get_it`, `fl_chart`, `pdf`/`printing` e `google_fonts`.
Ferramentas	Git/GitHub, Visual Studio Code, MySQL Workbench e XAMPP.

Tabela 5 – Tecnologias utilizadas.

2.3 Premissas

Disponibilidade da API e do Banco de Dados

Para o funcionamento do Auria, a AuriaAPI e o servidor MySQL precisam estar em execução e acessíveis pela rede, pois o aplicativo não guarda dados localmente além do token de acesso.

Disponibilidade de Acesso à Internet e Rede Local

É indispensável a conectividade com a internet ou com a rede corporativa interna para a sincronização imediata dos chamados, ordens de serviço e atualizações de estoque.

Dispositivos Compatíveis

Os usuários devem possuir computadores com Windows, navegadores atualizados ou smartphones Android para acessar o aplicativo e alimentar as ordens de serviço.

Configurações Sensíveis Fora do Código

A senha do banco de dados, a chave de assinatura do JWT e a senha do administrador inicial são configuradas por *user-secrets* (desenvolvimento) ou variáveis de ambiente (produção), nunca no código-fonte ou no repositório.

Treinamento das Equipes Operacionais e Técnicas

Assume-se que a equipe técnica e os administradores receberão capacitação prévia para o correto preenchimento de tempos, custos e consumo de peças.

Rotinas de Backup de Dados

Será estruturada uma política de backup do banco MySQL para assegurar a preservação do histórico de manutenções e das informações patrimoniais.

Colaboração e Engajamento dos Usuários

A eficácia do sistema depende do engajamento de todos os colaboradores na abertura imediata de chamados e no encerramento preciso das atividades por parte dos técnicos.

3 REQUISITOS

3.1 Requisitos Funcionais

RF01 - Autenticação e Controle de Acesso por Perfil

- O sistema deve permitir o login com e-mail e senha, devolvendo um token JWT válido por 8 horas, que o aplicativo envia em todas as requisições.
- O sistema deve oferecer três perfis: Solicitante, Técnico e Administrador, cada um com o seu menu e as suas permissões.
- Quando o token expirar, o aplicativo deve encerrar a sessão e voltar à tela de login.

RF02 - Gestão de Usuários

- O administrador deve poder cadastrar, editar e excluir usuários e definir o perfil de cada um.
- O cadastro público deve criar apenas usuários com perfil Solicitante.

RF03 - Cadastro de Ambientes e Ativos

- O sistema deve permitir o cadastro de ambientes com identificação, prédio ou bloco, andar e observações.
- O sistema deve permitir o cadastro de ativos com número de patrimônio, nome, estado de conservação, data de aquisição, prazo de garantia e vida útil estimada, vinculados a um ambiente.

RF04 - Abertura e Acompanhamento de Chamados

- O sistema deve permitir que o solicitante abra chamados informando o ativo, a descrição do problema, a localização detalhada, o tipo de manutenção (corretiva, preventiva ou preditiva) e a prioridade (baixa, média ou alta).
- O solicitante deve poder acompanhar o status dos próprios chamados e cancelá-los enquanto estiverem abertos.

RF05 - Aprovação de Chamados e Geração de Ordens de Serviço

- O administrador deve poder aprovar um chamado aberto designando um técnico, o que cria a ordem de serviço e muda o chamado para “Em Execução”.
- Na abertura da ordem de serviço, deve ser possível já informar as peças previstas, que são debitadas do estoque.

RF06 - Execução e Conclusão de Ordens de Serviço

- O técnico deve poder consultar as ordens designadas a ele, registrar as peças consumidas e concluir a ordem informando o tempo gasto, o custo de mão de obra e o parecer técnico.
- O sistema deve calcular o custo total da ordem de serviço (mão de obra mais peças) e concluir o chamado vinculado.

RF07 - Controle de Estoque de Peças e Materiais

- O sistema deve permitir o cadastro de itens com nome, código interno, quantidade, estoque mínimo e custo unitário.
- O sistema deve debitar automaticamente as peças consumidas nas ordens de serviço e listar os itens que estão no nível mínimo ou abaixo dele.

RF08 - Emissão de Notificações Automáticas

- A API deve notificar os administradores quando um novo chamado for aberto e quando um item de estoque atingir o nível mínimo.
- A API deve notificar o técnico quando uma ordem de serviço for designada a ele, e o solicitante quando o chamado entrar em execução, for concluído ou for cancelado pelo administrador.
- O usuário deve poder marcar notificações como lidas e excluí-las.

RF09 - Planejamento de Manutenção Preventiva

- O sistema deve apresentar o cronograma diário das ordens de serviço preventivas.
- O sistema deve listar alertas de ativos que precisam de manutenção preventiva, com o motivo de cada alerta.

RF10 - Indicadores e Relatórios Gerenciais

- O sistema deve disponibilizar um painel com o total de ordens, ordens concluídas e em aberto, taxa de conclusão, tempo médio de atendimento e de execução, custos de mão de obra e de peças, chamados por status e ativos com mais falhas.
- O painel deve permitir exportar um relatório em PDF.

3.2 Requisitos Não Funcionais

RNF01 - Segurança

- As senhas devem ser armazenadas com hash BCrypt, nunca em texto puro.
- O token JWT deve ser assinado com HMAC-SHA256, com chave de no mínimo 32 caracteres, e validado quanto à assinatura, ao emissor, ao público e à expiração em toda requisição.
- O acesso deve ser controlado por perfil (RBAC), tanto nos endpoints quanto nas regras de cada recurso, como o técnico designado ou o dono do chamado.
- Configurações sensíveis devem ficar fora do código-fonte e do repositório.

RNF02 - Desempenho

- A API deve usar cache em memória (estratégia *cache-aside*) nas consultas de leitura frequente ou de cálculo caro: lista de ambientes (5 minutos) e indicadores (2 minutos).
- O cache deve ser invalidado sempre que os dados correspondentes forem alterados, e cada situação (MISS, HIT, ARMAZENADO e INVALIDADO) deve aparecer no log da API.

RNF03 - Padronização e Tratamento de Erros

- Todas as respostas da API devem seguir o formato ResultWrapper, inclusive os erros de validação (400), de autenticação (401) e de permissão (403).
- Os erros devem ser convertidos em códigos HTTP adequados por um middleware central: regra de negócio (400), credenciais inválidas (401), acesso negado (403), recurso não encontrado (404), conflito (409) e erro interno (500, sem expor detalhes ao cliente).

RNF04 - Usabilidade e Acessibilidade

- A interface deve ser responsiva: o menu lateral vira gaveta em telas estreitas, e cadastros e detalhes abrem em painéis laterais sem sair da tela.
- O aplicativo deve seguir as diretrizes WCAG 2.2 (nível AA), com suporte a leitores de tela (TalkBack, VoiceOver e NVDA), navegação completa pelo teclado com foco visível e contraste mínimo de 4,5:1 nos textos.
- O usuário deve poder ajustar o tamanho do texto, ativar o alto contraste, usar uma fonte de leitura facilitada e reduzir as animações, com as preferências salvas no aparelho.
- A informação nunca deve depender apenas da cor: status, prioridade e nível de estoque devem sempre ter texto.

RNF05 - Manutenibilidade

- A API deve seguir a Clean Architecture, com as camadas Domain, Application, Infrastructure e API e a regra de dependência descrita na seção 2.2.3.
- O acesso a dados deve usar o padrão Repository com Unit of Work, e os controllers não devem conter regras de negócio.

RNF06 - Integridade e Validação dos Dados

- As entradas devem ser validadas com Data Annotations antes de chegar aos services, devolvendo a lista de erros na resposta.
- Operações que alteram vários dados (por exemplo, abrir uma ordem de serviço, debitar o estoque e notificar) devem ser gravadas em uma única transação.

RNF07 - Portabilidade

- O aplicativo deve rodar no Windows, na Web e no Android a partir do mesmo código Flutter.
- O endereço da API deve ser configurável em um único arquivo do aplicativo.

RNF08 - Documentação

- A API deve ser documentada pelo Swagger, e o repositório deve conter um README com a instalação, a configuração, os endpoints e exemplos de requisições, além do documento de decisões de arquitetura.

RNF09 - Privacidade e Conformidade

- O tratamento de dados pessoais deve seguir a LGPD: cada solicitante vê apenas os próprios chamados, cada usuário vê apenas as próprias notificações, e senhas nunca são devolvidas pela API.

3.3 Regras de Negócio

ID	Regra de Negócio
RN01	O cadastro público sempre cria um usuário com perfil Solicitante. Técnicos e administradores só podem ser criados por um administrador.
RN02	O e-mail de cada usuário deve ser único no sistema.
RN03	Todo chamado nasce com o status “Aberto”. Somente chamados abertos podem gerar ordem de serviço, e a partir daí passam a “Em Execução”.
RN04	Somente chamados com status “Aberto” podem ser cancelados, e apenas pelo administrador ou pelo próprio solicitante.
RN05	Não é possível abrir chamado para um ativo com estado de conservação “Desativado”.
RN06	A ordem de serviço só pode ser designada a um usuário com perfil Técnico ou Administrador.
RN07	Apenas o técnico designado (ou um administrador) pode registrar peças e concluir a ordem de serviço.
RN08	Uma ordem de serviço concluída não pode ser concluída novamente nem receber novas peças. Ao concluir a ordem, o chamado vinculado passa a “Concluído”.
RN09	A quantidade de uma peça consumida deve ser maior que zero e não pode ser maior que o saldo em estoque. Se a mesma peça for registrada mais de uma vez na mesma ordem, as quantidades são somadas.
RN10	Um item com saldo igual ou menor que o estoque mínimo é considerado crítico, e o consumo que o leva a esse nível gera notificação aos administradores.
RN11	Não é possível excluir: ambiente com ativos vinculados; ativo com chamados (deve ser marcado como “Desativado”); item de estoque já consumido em ordens de serviço; usuário com chamados ou ordens de serviço. Nesses casos, a API responde com conflito (409).
RN12	O administrador não pode excluir o próprio usuário nem remover o próprio perfil de administrador.
RN13	O solicitante só pode consultar os próprios chamados, e cada usuário só acessa as próprias notificações.
RN14	Um ativo entra em alerta de manutenção preventiva quando o estado de conservação é “Regular”, “Ruim” ou “Em Manutenção”, quando tem mais de 2 anos de uso ou quando a garantia está vencida. Ativos “Desativados” não geram alerta.
RN15	O custo total da ordem de serviço é a soma do custo de mão de obra com o custo das peças (quantidade multiplicada pelo custo unitário do item).

4 PLANO DE LIBERAÇÕES

Release 1 – Aplicativo Aura (1º semestre de 2026)

A primeira release entregou o aplicativo Aura em Flutter, com armazenamento local, nas iterações abaixo, já concluídas.

Iteração 1: Configuração Inicial e Cadastro de Ativos e Infraestrutura

- **Data Inicial:** 01/03/2026 **Data Final:** 15/03/2026
- **Objetivo:** Estabelecer as bases do projeto e implementar o inventário patrimonial, o mapeamento de ambientes e a gestão de responsáveis.

Iteração 2: Núcleo Operacional — Gestão de Chamados e Execução de OS

- **Data Inicial:** 16/03/2026 **Data Final:** 30/03/2026
- **Objetivo:** Implementar a abertura de chamados e a execução das ordens de serviço.

Iteração 3: Automatização Preventiva e Integração de Estoque

- **Data Inicial:** 01/04/2026 **Data Final:** 15/04/2026
- **Objetivo:** Agregar o planejamento preventivo e integrar o estoque ao consumo das ordens de serviço.

Iteração 4: Indicadores Gerenciais (BI), Notificações e Implantação

- **Data Inicial:** 16/04/2026 **Data Final:** 30/04/2026
- **Objetivo:** Consolidar o painel analítico e as notificações e realizar os testes finais.

Release 2 – AuriaAPI e Integração (2º semestre de 2026)

A segunda release transforma o Aura em uma solução cliente-servidor, com a AuriaAPI em ASP.NET Core e o aplicativo integrado a ela.

Iteração 5: Fundação da AuriaAPI

- **Data Inicial:** 01/09/2026 **Data Final:** 16/09/2026
- **Objetivo:** Criar a API REST com as entidades do sistema, o banco MySQL e os requisitos técnicos da disciplina.

- **Atividades:**

    1. **Estrutura do Projeto:** Criar a solução ASP.NET Core organizada em Clean Architecture e o repositório no GitHub.
    2. **Banco de Dados:** Mapear as entidades com Entity Framework Core e gerar a migration inicial do MySQL.
    3. **Segurança:** Implementar o login com JWT, o hash de senhas com BCrypt e a restrição de acesso por perfil.
    4. **Padronização e Cache:** Implementar o ResultWrapper em todas as respostas e o cache em memória com logs de MISS, HIT e invalidação.

Iteração 6: Integração com o Aplicativo Flutter

- **Data Inicial:** 17/09/2026 **Data Final:** 29/09/2026
- **Objetivo:** Fazer o aplicativo consumir a API no lugar do armazenamento local.
- **Atividades:**

    1. **Cliente HTTP:** Criar o cliente que envia o token, interpreta o ResultWrapper e trata a expiração da sessão.
    2. **Endpoints de Suporte:** Criar os endpoints de ordens de serviço, consumo de estoque e notificações usados pelo aplicativo.
    3. **Documentação:** Escrever o README com instalação, configuração e exemplos de requisições.

Iteração 7: Refatoração e Regras de Negócio na API

- **Data Inicial:** 30/09/2026 **Data Final:** 15/10/2026
- **Objetivo:** Concluir os requisitos do trabalho e levar as regras de negócio do aplicativo para a API.
- **Atividades:**

    1. **Clean Architecture:** Separar as camadas Domain, Application, Infrastructure e API, com repositórios e Unit of Work.
    2. **Regras na API:** Mover para a API a baixa de estoque, as notificações, os indicadores e os alertas de preventiva.
    3. **Configurações Sensíveis:** Retirar senhas e chaves do código, usando *user-secrets*.

Iteração 8: Nova Interface, Acessibilidade e Entrega

- **Data Inicial:** 16/10/2026 **Data Final:** 30/10/2026
- **Objetivo:** Entregar a nova interface do aplicativo, a acessibilidade e a documentação final.
- **Atividades:**

    1. **Novo Design:** Criar a interface por perfil (Administrador, Técnico e Solicitante), com temas claro e escuro e o novo logo do Auria.
    2. **Acessibilidade:** Implementar o suporte a leitores de tela, a navegação por teclado e o painel de ajustes (tamanho do texto, alto contraste, fonte legível e redução de animações).
    3. **Dados de Demonstração:** Criar os dados de exemplo para testes e apresentação.
    4. **Testes e Entrega:** Executar o roteiro completo de testes, atualizar a documentação e publicar a versão final no GitHub.